HANDS-ON LAB 03 GUIDE & COPY-PASTE PROMPTS • TELECOMMUNICATIONS EDITION (TELECOM NEXUS)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (EN)

Lab 03: Live OSS/BSS Telemetry via FastMCP, Lifecycle Hooks & Gemini Enterprise Security

Duration: 50 minutes

Delivered by: **Tryolabs** (Powered by Google Antigravity & Gemini Enterprise)

Antigravity Capabilities Practiced: Model Context Protocol (**`mcp_config.json`** + FastMCP), Lifecycle Hooks (**`hooks.json`** **`PreToolUse`** & **`PostToolUse`**), Antigravity Client Security Settings (Sandbox, Tool Execution Policy, Allowlists/Denylists), and Google Cloud **Gemini Enterprise** Governance.

Language / Idioma: English | Español • **Prompt Playbook:** PROMPTS.md | PROMPTS.es.md • **PDF:** EN | ES

1. Scenario & Enterprise Security Architecture

Telecom operators manage highly sensitive infrastructure (3GPP 5G RAN gNodeBs, Core Charging Functions) and regulated Customer Proprietary Network Information (**CPNI**: **`IMSI`**, **`MSISDN`**, **`IMEI`**). Connecting AI agents to Telecom OSS/BSS databases requires two layers of enterprise governance:

Layer 1: Google Cloud & Gemini Enterprise Governance (Cloud Plane)

When attendees authenticate Antigravity with their corporate **Gemini Enterprise** license:

1. **IAM & Seat Governance**: Access is governed via Google Cloud IAM (**`roles/cloudaicompanion.user`**) federated with corporate SSO (Workspace / Cloud Identity / Okta / Entra ID).
2. **Cloud Audit Logs (`cloudaicompanion.googleapis.com`)**: Platform administrators monitor **`Admin Activity`** and **`Data Access`** audit logs in **Google Cloud Logging** for compliance and license utilization—without logging proprietary source code or subscriber payloads.
3. **Zero Training & Perimeter Security**: Customer code, prompts, and 3GPP/BSS telemetry are **never** used to train foundation models, are covered by Google Cloud IP indemnification, and can be perimeter-restricted with **VPC Service Controls (VPC-SC)**.

Layer 2: Antigravity Client & Workspace Guardrails (Local Plane)

Inside **Antigravity 2.0** and **Antigravity IDE** (**`Settings`** -> **`Global`** & **`Project-Level Settings`**):

1. **Tool Execution Policy**: Choose between **`strict`**, **`request-review`**, **`proceed-in-sandbox`**, or **`always-proceed`**.
2. **Terminal Sandbox**: Isolates shell commands executed by the agent inside an OS-level sandbox.

3. **File & Internet Access Policies**: Restrict **`Non-Workspace File Access`** (**deny**/**ask**) and **`Internet Access Policy`** (**deny**/**ask**), plus configure **`Command Allowlist / Denylist`** and **`Browser Allowlist`**.
4. **Deterministic Lifecycle Hooks (`.agents/hooks.json`)**: Intercept every tool call (**`PreToolUse`** / **`PostToolUse`**) via **`scripts/audit_hook.py`** to block destructive SQL (**`DROP TABLE`**, **`TRUNCATE`**, **`DELETE FROM b2b_contracts`**) before execution!

2. Lab Setup

Run the Lab 03 installer (which automatically applies Labs 01 and 02 solutions first):

```
./labs/lab-03-mcp-and-hooks/install.sh
```

3. Step-by-Step Instructions

Step 3.1: Complete the 4 FastMCP Tools in `mcp_server/telecom_mcp.py`

Open mcp_server/telecom_mcp.py. **`query_ran_cell_health`** is already implemented. Use Antigravity (see PROMPTS.md) to implement the remaining three **`@mcp.tool()`** functions:

1. **`calculate_contract_penalty_exposure(contract_id: str | None = None)`**
2. **`inspect_unbilled_cdrs(contract_id: str | None = None, roaming_only: bool = False)`** — **Important**: Mask **`imsi`** and **`msisdn`** as **`masked_imsi`** and **`masked_msisdn`** (e.g., **`3104***0001`**) so raw CPNI is never exposed!
3. **`evaluate_slice_capacity(target_tac: str, requested_slice_sst: int, requested_bandwidth_mbps: int, max_latency_ms: float)`**

Step 3.2: Register the MCP Server in `.agents/mcp_config.json`

Create **`.agents/mcp_config.json`**:

```
{
  "mcpServers": {
    "telecom-nexus-mcp": {
      "command": "./.venv/bin/python",
      "args": ["mcp_server/telecom_mcp.py"],
      "env": {
        "PYTHONUNBUFFERED": "1"
      }
    }
  }
}
```

Step 3.3: Configure `PreToolUse` & `PostToolUse` Hooks in `.agents/hooks.json`

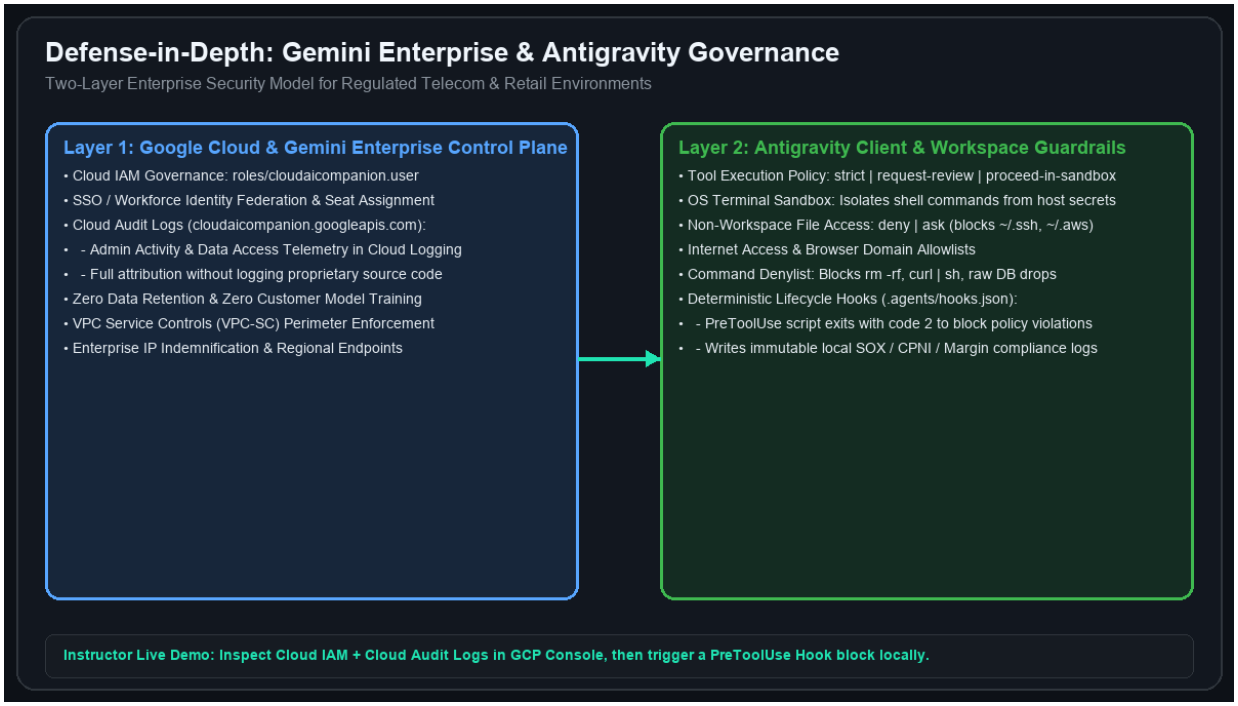
Inspect scripts/audit_hook.py and create **`.agents/hooks.json`** so every **`run_command`** step is validated by **`audit_hook.py`** before execution and logged to **`data/logs/hook_audit.log`** afterward.

4. Verification

Run the Lab 03 verification script:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Need the Reference Solution? Run `./labs/lab-03-mcp-and-hooks/apply_solution.sh`.



LAB 03 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 03 — Optimized Antigravity Prompts Cheat Sheet (English)

Language / Idioma: English | Español • **Lab Guide:** README.md | README.es.md • **PDF:** EN | ES

Goal: Connect **Google Antigravity** (governed by **Gemini Enterprise** and delivered by **Tryolabs**) to live Telecom OSS/BSS databases via **Model Context Protocol (FastMCP)** and enforce deterministic CPNI/database security via **Lifecycle Hooks (`hooks.json`)**.

1. Discovery Prompt (@ Mentions)

```
@mcp_server/telecom_mcp.py @scripts/audit_hook.py @tests/test_lab03_mcp_hooks.py
Inspect the starter FastMCP server in `mcp_server/telecom_mcp.py`, the compliance hook v \
  alidator in `scripts/audit_hook.py`, and the test expectations in `tests/test_lab03_mc \
  p_hooks.py`. List the 3 unimplemented MCP tools in `telecom_mcp.py` and the exact sche \
  mas required for `.agents/mcp_config.json` and `.agents/hooks.json`.
```

2. Planning Prompt (`/plan`)

```
/plan Create an implementation plan for Lab 03 to:
1. Complete `mcp_server/telecom_mcp.py` by implementing:
   - `calculate_contract_penalty_exposure(contract_id: str | None = None)` using `evalua \
  te_contract_sla_compliance()` and `Decimal`.
   - `inspect_unbilled_cdrs(contract_id: str | None = None, roaming_only: bool = False)` \
   using `rate_cdr_record()` and masking `imsi` (`masked_imsi`) and `msisdn` (`masked_ms \
  isdn`) with `***` for CPNI compliance.
   - `evaluate_slice_capacity(target_tac: str, requested_slice_sst: int, requested_bandw \
  idth_mbps: int, max_latency_ms: float)` returning `READY_TO_ACTIVATE`, `REQUIRES_SLICE \
  _REMEDIATION`, or `REJECTED`.
2. Create `.agents/mcp_config.json` registering `telecom-nexus-mcp` over Stdio (`./.venv \
  /bin/python mcp_server/telecom_mcp.py`).
3. Create `.agents/hooks.json` wiring `PreToolUse` (`matcher: "run_command"`) and `PostT \
  oolUse` (`matcher: "run_command|edit_file|write_file"`) to `scripts/audit_hook.py`.
```

3. Instructive Inline Edit Prompt (`Cmd+I` / `Ctrl+I`)

Open **`mcp_server/telecom_mcp.py`**, highlight the three stubbed **`@mcp.tool()`** functions, and press **`Cmd+I`** / **`Ctrl+I`**:

```
Implement `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs` (masking IMSI a \
  nd MSISDN with `***` in `masked_imsi` and `masked_msisdn`), and `evaluate_slice_capaci \
  ty` using `src.services.sla_monitor`, `src.services.cdr_rating`, and `src.database.fet \
  ch_all` so all assertions in `tests/test_lab03_mcp_hooks.py` pass.
```

4. Autonomous Implementation & Verification Prompt

```
Implement the 3 remaining MCP tools in @mcp_server/telecom_mcp.py, create `.agents/mcp_c \
  onfig.json` and `.agents/hooks.json`, and run `./labs/lab-03-mcp-and-hooks/verify.sh`.
```

5. Testing Live MCP Tools & Security Hooks Prompt

After **`verify.sh`** passes, test the MCP tools and the **`PreToolUse`** safety hook directly from Antigravity Chat:

```
1. Use our Telecom MCP tools (or call the functions in `mcp_server/telecom_mcp.py`) to i \
  nspect `TAC-NY-001` slice `SST=2`, calculate penalty exposure for `CTR-B2B-1002`, and  \
  inspect unbilled roaming CDRs with CPNI-masked IMSIs.
2. Test our `PreToolUse` lifecycle hook by simulating a call to `scripts/audit_hook.py - \
  -event PreToolUse` with a destructive SQL payload (`DROP TABLE b2b_contracts;`) and sh \
  ow how the hook returns `"decision": "deny"` and logs the event in `data/logs/hook_aud \
  it.log`.
```

6. Power-User & Security Governance Prompts (`/boost`, `/grill-me`, `/schedule`)

Deep Security Architecture Review (`/boost`)

```
/boost Perform a comprehensive Telecom OSS/BSS threat-modeling review combining our loca \
  l Antigravity `.agents/hooks.json` guardrails, Terminal Sandbox settings, and Google C \
  loud Gemini Enterprise governance (`roles/cloudaicompanion.user`, Cloud Audit Logs for \
   `cloudaicompanion.googleapis.com`, and VPC Service Controls).
```

Continuous SLA & Hook Audit Monitoring (`/schedule`)

```
/schedule Every 10 minutes, run `query_ran_cell_health` across all 5 TACs and check `dat \
  a/logs/hook_audit.log` for any denied tool executions.
```